

Input Order Is an Uncontrolled Variable in Lossless Point Cloud Compression Benchmarks

September 20, 2026

Abstract

Lossless point cloud codecs are compared at equal bitrate, but they do not all encode the same object. Some reconstruct the multiset of points; others reconstruct the sequence. The arrangement of the input is never part of the test conditions, so a bitrate reported without it is underdetermined. We make the arrangement an explicit factor: eight order conditions applied to the same point set, four lossless coders, 26 blocks drawn from 12 files, 204 round-trip verified measurements.

The effect is large and coder-dependent. Under a uniform random permutation the geometry code length of LASzip rises by 121.9% and that of a sequential context coder by 130.9% (medians), while a G-PCC configuration agrees with the identity order to three significant digits in all 204 rows. Two null controls are needed, not one: reversing the input is a structured treatment whose spread (-1.28% to $+1.59\%$ for one coder) is five times the seed-to-seed noise, and using the smaller control alone inflates the count of blocks that appear to respond from 6 to 15.

We also report what the data do *not* support. A codec that derives its coding order from a sort key is order-invariant only while that key is not fully degenerate: with a completely degenerate key its cost rises by 509.7 to 787.4% and it collapses onto a plain geometry coder. But the threshold is not injectivity, as we first claimed and as our own data refute: a file whose key is 82.4% duplicated moves by only 7.0%, and one at 45.0% by 0.06%. With one sample between 45% and 100% we cannot say whether the boundary is graded or abrupt. We conclude with the minimum an order specification must fix for lossless comparisons to mean anything.

1 Introduction

A lossless point cloud codec is judged by bits per point. The comparison assumes that the coders under test are solving the same problem. They are not.

An octree coder such as G-PCC reconstructs the *multiset* of occupied positions. The order in which the points arrived is discarded, and the decoder is free to emit them in whatever order the traversal produces. A sequential coder such as LASzip reconstructs the *sequence*: point i of the output is point i of the input. For $n = 10^6$ the difference is $\log_2(n!)/n = 18.489$ bits per point of information that one coder carries and the other does not. That is comparable to the entire geometry budget of an airborne LiDAR tile.

This is known in the abstract. What is missing is a measurement. The common test conditions used to compare lossless point cloud coders do not specify the arrangement of the input, and no published study varies it systematically. A bitrate reported without an order condition is therefore not reproducible: the same coder on the same points can differ by more than a factor of two depending on how the file happened to be written.

We treat the arrangement as an experimental factor. Section 3 defines eight order conditions and, importantly, two null controls. The second control is the part that is easy to get wrong, and we got it wrong first: reversing the input is not noise, it is a treatment, and its spread sets the resolution of the whole experiment.

1.1 Contributions

1. A protocol that makes input order an explicit factor in lossless point cloud comparison, with two null controls and a stated resolution (Section 3).
2. A measurement of order sensitivity across four coders, 26 blocks and eight conditions, every row verified by round trip (Section 5).
3. A negative result about our own earlier claim: order invariance of a key-sorted coder is governed by full degeneracy of the key, not by its injectivity, and our data refute the stronger statement (Section 6).
4. An account of what the correlational part of the evidence cannot support at this sample size, and what the two endpoints can (Section 8).

2 Related work

Order-invariant geometry coding is standard. The octree of G-PCC and the kd-tree of Draco both code a multiset, so their geometry cost does not depend on the input arrangement. TMC13 additionally offers `--sortInputByAzimuth` and `--predGeomSort`, which re-derive an acquisition-like order internally, and a `canonical_point_order_flag`. Re-deriving a scan order from the data is therefore prior art, not a contribution.

Sending the order as a separate channel is patented. US10796457B2 (Van Beek, Intel, priority

Table 1: Lossless coders divide on two independent axes. The comparison at equal bitrate is only meaningful within a row.

	Cost depends on order	Cost independent
Multiset	—	G-PCC
Sequence	LASzip, ours (v3)	ours (scan), cond.

2018) states that if point positions are coded losslessly the reconstructed X, Y, Z are identical whatever the coding order, and that the choice of coding order may be transmitted so that the original order can be reconstructed. The combination we measure is within the scope of that disclosure.

The $\log(n!)$ framework is established in the opposite direction. Shuffle coding [2] and Random Order Coding [3] spend effort to *save* the $\log(n!)$ bits of an unordered set. We are measuring what it costs to *keep* them. The machinery is the same with the sign reversed, and neither is novel here.

LASzip [1] is the representative order-dependent coder: it does not re-sort, and its predictor follows the scan line. Re-sorting is left to external tools, which destroy the input order.

What we could not find in the literature is the measurement itself: a systematic comparison of code length under controlled input orders, with a null control, for lossless point cloud coders. Our own earlier work [4] reports a single instance of the effect (one file, 2M points, one direction) and uses it to argue that reordering is not a levelling operation. The present paper turns that remark into a protocol and finds that the earlier reading was too simple.

3 The protocol

3.1 Two axes, not one

Table 1 separates two properties that are usually conflated: what the decoder is required to reconstruct, and whether the code length depends on the arrangement it was given. A coder in the top row is not comparable at equal bitrate with one in the bottom row, because the bottom row is also transmitting the permutation.

3.2 Order conditions

For each block of n points we hold the point set fixed and vary only the arrangement:

- **identity** — the order in the source file (the baseline);
- **reverse** — the exact reversal;
- **Morton** — sorted by Morton code of the coordinates, each axis normalised to its own range before interleaving;
- **window** w — a uniform random permutation applied inside consecutive windows of w points, for $w \in \{10^2, 10^3, 10^4\}$, kept only when $10w \leq n$;

- **random** — a uniform random permutation of all n points, run twice with different seeds.

The window conditions interpolate between the identity and a uniform permutation, so a coder whose cost tracks local disorder should respond monotonically.

3.3 Two null controls

A measurement of this kind needs a floor below which a difference cannot be distinguished from the apparatus. We use two controls, and the distinction matters.

(a) *Seed-to-seed*. The two random conditions differ only in the seed. Their difference measures variation within a single treatment, with the baseline cancelling.

(b) *Reverse*. Reversal preserves every local neighbourhood; a sequential predictor that is symmetric in direction should be unaffected. It is nevertheless a structured rearrangement, and its deviation from zero measures what a *benign* rearrangement costs.

We initially used (b), replaced it with (a) in a second design, and were wrong to do so. Control (b) is the larger of the two by a factor of five for one coder, and it is the one that bounds what we can claim. The effective floor is the larger of the two.

3.4 Coders

Four lossless coders, all measured on geometry only so that the levels are comparable:

- **G-PCC** — TMC13 **release-v23.0-rc2**, lossless octree, geometry only;
- **LAZ** — LASzip via a point record containing only X, Y, Z ;
- **v3** — our sequential context coder over the three coordinate columns, using cross-axis context;
- **scan** — our scan-model coder, which sorts internally by the key (flight line, timestamp) before predicting.

3.5 Verification

Every reported number comes from a run that was decoded and compared. For our own coder the candidate is pinned, so the configuration that is reported is the configuration that was verified, and the fallback path that would silently substitute an embedded container is disabled. All 204 rows pass. The meaning of “verified” differs by coder and we state it rather than eliding it: for LAZ and for our container it is equality of all columns *in order*; for G-PCC it is equality of the multiset of coordinates after a lexicographic sort.

4 Experiments

Twelve files from airborne LiDAR (AHN3/4/5 tiles, Autzen, Red Rocks), terrestrial and structured-light sources, and synthetic cases. Seven large files contribute three blocks of 10^6 points each, below the default `sliceMaxPoints` of TMC13 so that G-PCC codes

Table 2: Code length relative to the identity order, in percent. Median [interquartile range] over 26 blocks.

Condition	G-PCC	LAZ	v3	scan
reverse	+0.0	−0.0	−0.0	−0.1
Morton	+0.0	+16.1	+23.0	−0.0
window 10^2	+0.0	+65.5	+70.7	+0.3
window 10^3	+0.0	+80.8	+86.0	+0.3
window 10^4	+0.0	+89.1	+95.6	+0.2
random 1	+0.0	+121.9	+130.9	+0.2
random 2	+0.0	+121.9	+130.9	+0.2
IQR, random	—	[89, 140]	[102, 153]	[0.0, 1.5]

a single slice; five small files are used whole (10,683 to 110,000 points). This gives 26 blocks. With eight conditions, less the four window conditions that the small blocks cannot support, $26 \times 8 - 4 = 204$ rows. The four coders are columns, not a factor.

The three blocks within a large file give, for the first time in this line of work, a spread for a quantity that was previously a single sample.

5 Results

5.1 The effect is large and coder-dependent

Table 2 is the main result. A uniform permutation of the same points costs LASzip 121.9% and our sequential coder 130.9% of their identity-order geometry budget. G-PCC agrees with the identity order to three significant digits in every one of the 204 rows, as its construction requires; at 10^6 points three digits is a resolution of 0.001 bpp, or 125 bytes, so the correct statement is “agrees to three digits”, not “exactly zero”.

The interquartile range is wide. Reporting only the median would hide it, and the spread within a single file is itself large: the three blocks of one airborne file move between +38.6% and +129.0% for the same coder and condition.

5.2 The window conditions separate two behaviours

Code length increases monotonically with the window width for LAZ (22 of 22 blocks that support all four conditions) and for v3 (22 of 22). For the scan coder it is monotone in only 3 of 22 — and those three are the three blocks of the single file whose key is fully degenerate, where the response is not noise but the largest effect in the study ($+225 \rightarrow +317 \rightarrow +413 \rightarrow +522\%$). The remaining 19 blocks are flat at the level of the floor.

This asymmetry is the useful part. Where the scan coder is invariant, it is invariant at every window width; where it is not, it degrades gradually with disorder like the sequential coders do.

5.3 Morton order is not uniformly bad

Sorting into Morton order raises the median cost by 16.1% (LAZ) and 23.0% (v3), which reproduces the di-

Table 3: Two null controls, in percentage points. (a) seed-to-seed difference between the two random conditions; (b) deviation of the reverse condition from zero.

	(a) med.	(a) max	(b) min	(b) max	floor
G-PCC	0.000	0.000	+0.000	+0.000	0.000
LAZ	0.009	0.664	−0.385	+0.588	0.664
v3	0.015	0.261	−1.099	+0.070	1.099
scan	0.015	0.305	−1.282	+1.591	1.591

Table 4: Cost of a uniform permutation, by file, against the duplication rate of the sort key. Representative block per file; range over blocks in parentheses.

File	key dup.	LAZ	v3	scan
red-rocks	100.0	+458	+513	+517
simple1_4	100.0	+829	+642	+787
plane	82.4	+189	+209	+7.0
workshop	39.2	+133	+146	+1.8
fullwave	33.7	+69	+86	+0.5
AHN3	24.0	+123	+134	+0.7
AHN4 (b)	18.3	+136	+149	−1.1
AHN5	15.5	+116	+131	+0.2
autzen-trim	9.7	+119	+128	−0.4
AHN4 (a)	8.3	+124	+135	−0.2
autzen-2023	7.3	+84	+87	+0.0
vegetation	0.0	+56	+56	+0.0

rection of the single measurement in [4]. The magnitudes are not comparable with that earlier figure of 27.1%, which was one airborne file of 2M points with a different column definition.

The sign is not uniform. Morton order *reduces* the cost in 6 of 26 blocks for LAZ, 5 of 26 for v3 and 14 of 26 for the scan coder, with a range for v3 of $−21.0\%$ to $+232.3\%$. “Space-filling-curve order is bad for sequential predictors” is a statement about medians, not a rule.

5.4 The floor

Table 3 is the methodological point of the paper. For the scan coder the benign rearrangement moves the code length five times as far as the seed does. If the seed difference alone is taken as the floor, 15 of 26 blocks appear to respond to the order; against the effective floor, 6 do. The 9 blocks that are gained by the smaller floor all fall inside the range that reversal itself spans, so they cannot be separated from a rearrangement that costs nothing.

We report this because we made the substitution ourselves, in a redesign that was otherwise an improvement, and it took an outside reading to catch it.

6 Order invariance and the sort key

The scan coder sorts internally by (flight line, timestamp) before predicting. When that key is a total order on the points, the sort undoes any input permutation and the code length cannot depend on it. Table 4 shows what happens when it is not.

What the endpoints show. With a fully degenerate key the coder loses the property completely: +509.7 to +787.4% over the four such blocks, and on red-rocks the scan coder (+517%) lands on the plain geometry coder (+513%) — it has collapsed into one. At the other end, the file whose key has no duplicates is identical to three digits across all seven of its conditions. These two facts establish the dependence without any appeal to a correlation.

What we claimed and must withdraw. We wrote, twice, that injectivity of the key is what decides the property. Our own data refute it. The file `plane` has an 82.4% duplicated key — far from injective — and moves by only 7.0%; one block at 45.0% moves by 0.06%. What the data separate is *full degeneracy* from everything else, not injective from non-injective. Stated as injectivity the condition is simultaneously too strong (only one of the eleven files in the invariant cell satisfies it) and too weak (ten non-injective files sit in that cell).

What the shape of the response suggests. The degenerate blocks grow with the window width; `plane` saturates (+6.86, +7.06, +7.01% for window 10^2 , 10^3 and random). A graded mechanism and a saturating one are not the same mechanism at different strengths. With a single sample between 45% and 100% duplication we cannot decide the shape of the boundary.

The correlation does not carry weight here. Rank correlation between key duplication and the cost of a permutation is +0.697 ($p = 0.025$, $n = 10$ non-degenerate files) for the scan coder. There are nine such tests in the paper; the Bonferroni bound is $p \times 9 = 0.23$. Removing any one of four files from the ten destroys significance. We report the correlation for completeness and do not build on it.

7 What this means for benchmarks

The order must be part of the test conditions. A lossless bitrate reported without an order condition is underdetermined by up to a factor of two for a sequential coder. Common test conditions that fix the sequences, the quantisation and the attribute set, but not the arrangement, do not fix the number they are comparing.

Reordering the input is not a levelling operation. It is tempting to sort every input into a canonical order so that all coders face the same thing. That destroys the premise of the order-dependent coders instead of neutralising an advantage: the identity order is where an acquisition-order coder is designed to work, and Morton order is not neutral either (Section 5).

The comparison must be within a row of Table 1. If a multiset coder is compared with a sequence coder, the $\log_2(n!)/n = 18.489$ bits per point that only one of them carries have to be accounted for on one side or the other. We note that this figure applies to the 21 blocks of 10^6 points and not to the five smaller ones, and that it is the information content of the permutation, not the cost a coder pays for disorder; the two are not directly comparable.

A minimum specification. An order clause for lossless point cloud test conditions needs to fix: (i) which arrangement is the baseline, (ii) whether coders may re-sort internally and whether the original order must be recoverable, (iii) how the permutation is accounted for when the two sides of a comparison differ on (ii), and (iv) a null control with a stated resolution, so that a reported difference can be separated from what a benign rearrangement costs.

8 Limitations

Sample size. Twelve files, of which four airborne tiles share a footprint and two share a site: eight independent sites, six among the non-degenerate. A power calculation for $\rho = 0.7$ at 80% needs about 13. The correlational part of this study is underpowered and we do not rest any conclusion on it. The two endpoints of Section 6 do not need it.

The gap in the key axis. No data between 45% and 82% duplication. The shape of the boundary is open.

Variance. The three blocks of a large file give a within-file spread but not a between-file one; the five small files remain single blocks, and the correlation still has one point per file.

Accounting asymmetry. The scan coder’s figure covers X, Y, Z only, and those columns cannot be decoded without the timestamp and flight line that form the key. Adding the side columns raises the cost of a permutation to +137.9% in a separate measurement. The correct statement is conditional: *given* the side columns, the geometry cost of the scan coder is order-independent while the key is not fully degenerate.

Scope. LAS-family files only. No automotive (KITTI-style) or terrestrial laser scanning formats, and one codec family per row of Table 1.

Novelty. Every component of the invariance property is prior art (Section 2). What is new here is the measurement and the protocol, together with the finding that the property is governed by degeneracy of the key and the quantification of what happens when the key degenerates.

9 Conclusion

Input order is a factor with an effect size of more than 100% on sequential lossless point cloud coders, it is absent from the test conditions used to compare them, and it interacts with coder internals in ways that are not monotone. Making it explicit costs one extra axis in the experimental design and two null controls. Leaving it implicit makes the reported numbers irreproducible.

Our own experience is part of the argument. Between the first and second version of this experiment we replaced the larger null control with the smaller one, narrowed a dispersion summary in a way that lost information, compared counts across a changed denominator, and stated a condition (injectivity) that our data contradict. Each of these made the result look cleaner.

None was caught by re-reading; all were caught by an outside reading of the data against the text.

References

- [1] M. Isenburg, “LASzip: lossless compression of LiDAR data,” *Photogrammetric Engineering and Remote Sensing*, 79(2), 2013.
- [2] J. Kunze et al., “Entropy coding of unordered data structures,” *ICLR*, 2024.
- [3] D. Severo et al., “Compressing multisets with large alphabets,” *IEEE Journal on Selected Areas in Information Theory*, 2022.
- [4] Lossless coding of airborne LiDAR attributes in a geometry-derived order. Companion manuscript.